SET

In [2]:
```
thisset={"apple","banana","cherry"}
print(thisset)
```

```
{'banana', 'cherry', 'apple'}
```

In [4]:
```
thisset={"apple","banana","cherry","apple"}
print(thisset)
```

```
{'banana', 'cherry', 'apple'}
```

In [5]:
```
thisset={"apple","banana","cherry",True,1,2}
print(thisset)
```

```
{True, 2, 'banana', 'apple', 'cherry'}
```

In [6]:
```
thisset={"apple","banana","cherry",False,True,0}
print(thisset)
```

```
{False, True, 'banana', 'apple', 'cherry'}
```

In [7]:
```
thisset={"apple","banana","cherry"}
print(len(thisset))
```

```
3
```

In [8]:
```
myset={"apple","banana","cherry"}
print(type(myset))
```

```
<class 'set'>
```

In [9]:
```
thisset=set(("apple","banana","cherry"))
print(thisset)
```

```
{'cherry', 'banana', 'apple'}
```

In [10]:
```
thisset={"apple","banana","cherry"}
for x in thisset:
    print(x)
```

```
banana
cherry
apple
```

In [11]:
```
thisset={"apple","banana","cherry"}
print("banana" in thisset)
thisset={"apple","banana","cherry"}
print("banana" not in thisset)
```

```
True
False
```

In [12]:
```
thisset={"apple","banana","cherry"}
thisset.add("orange")
```

```
print(thisset)
```

```
{'banana', 'cherry', 'orange', 'apple'}
```

In [13]:
```
thisset={"apple","banana","cherry"}
tropical={"pineapple","mango","papaya"}
thisset.update(tropical)
print(thisset)
```

```
{'banana', 'mango', 'pineapple', 'papaya', 'apple', 'cherry'}
```

In [16]:
```
thisset={"apple","banana","cherry"}
mylist=["kiwi","orange"]
thisset.update(mylist)
print(thisset)
```

```
{'cherry', 'orange', 'banana', 'apple', 'kiwi'}
```

In [17]:
```
thisset={"apple","banana","cherry"}
thisset.discard("banana")
print(thisset)
```

```
{'cherry', 'apple'}
```

In [18]:
```
thisset={"apple","banana","cherry"}
x=thisset.pop()
print(x)
print(thisset)
```

```
banana
{'cherry', 'apple'}
```

In [19]:
```
thisset={"apple","banana","cherry"}
del thisset
```

In [21]:
```
thisset={"apple","banana","cherry"}
for x in thisset:
    print(x)
```

```
banana
cherry
apple
```

In [22]:
```
set1={"a","b","c"}
set2={1,2,3}
set3=set1.union(set2)
print(set3)
```

```
{1, 'a', 2, 3, 'c', 'b'}
```

In [23]:
```
set1={"a","b","c"}
set2={1,2,3}
set3=set1|set2
print(set3)
```

```
{1, 'a', 2, 3, 'c', 'b'}
```

In [24]:
```
set1={"a","b","c"}
set2={1,2,3}
set3={"john","elena"}
set4={"apple","bananas","cherry"}
mylist=set1.union(set2,set3,set4)
print(myset)
```

```
{'banana', 'cherry', 'apple'}
```

In [25]:
```
set1={"a","b","c"}
set2={1,2,3}
set3={"john","elena"}
set4={"apple","bananas","cherry"}
myset=set1|set2|set3|set4
print(myset)
```

```
{1, 2, 3, 'elena', 'b', 'cherry', 'bananas', 'a', 'c', 'john', 'apple'}
```

In [26]:
```
x={"a","b","c"}
y=(1,2,3)
z=x.union(y)
print(z)
```

```
{1, 2, 3, 'b', 'a', 'c'}
```

In [28]:
```
set1={"a","b","c"}
set2={1,2,3}
set1.update(set2)
print(set1)
```

```
{1, 'a', 2, 3, 'c', 'b'}
```

In [29]:
```
set1={"apple","banana","cherry"}
set2={"google","microsoft","apple"}
set3=set1&set2
print(set3)
```

```
{'apple'}
```

In [30]:
```
set1={"apple","banana","cherry"}
set2={"google","microsoft","apple"}
set1.intersection_update(set2)
print(set1)
```

```
{'apple'}
```

In [31]:
```
set1={"apple",1,"banana",0,"cherry"}
set2={False,"google",1,"apple",2,True}
set3=set1.intersection(set2)
print(set3)
```

```
{False, 1, 'apple'}
```

In [32]:
```
set1={"apple","banana","cherry"}
set2={"google","microsoft","apple"}
set3=set1.difference(set2)
```

```
print(set3)
```

```
{'banana', 'cherry'}
```

In [33]:
```
set1={"apple","banana","cherry"}
set2={"google","microsoft","apple"}
set3=set1-set2
print(set3)
```

```
{'banana', 'cherry'}
```

In [34]:
```
set1={"apple","banana","cherry"}
set2={"google","microsoft","apple"}
set1.difference_update(set2)
print(set1)
```

```
{'banana', 'cherry'}
```

In [35]:
```
set1={"apple","banana","cherry"}
set2={"google","microsoft","apple"}
set3=set1^set2
print(set3)
```

```
{'google', 'cherry', 'banana', 'microsoft'}
```

In [36]:
```
set1={"apple","banana","cherry"}
set2={"google","microsoft","apple"}
set1.symmetric_difference_update(set2)
print(set1)
```

```
{'banana', 'cherry', 'microsoft', 'google'}
```

In [37]:
```
x=frozenset({"apple","banana","cherry"})
print(x)
print(type(x))
```

```
frozenset({'banana', 'cherry', 'apple'})
<class 'frozenset'>
```

DICTIONARY

In [38]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
print(thisdict)
```

```
{'brand': 'ford', 'model': 'mustang', 'year': 1964}
```

In [39]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
print(thisdict["brand"])
```

```
ford
```

In [40]:
```
thisdict={"brand":"ford","model":"mustang","year":1964,"year":2020}
print(thisdict)
```

```
{'brand': 'ford', 'model': 'mustang', 'year': 2020}
```

In [41]:
```
print(len(thisdict))
```

```
3
```

In [43]:
```
thisdict={"brand":"ford","model":"mustang","year":1964,"colors":["red","white"
print(thisdict)
```

```
{'brand': 'ford', 'model': 'mustang', 'year': 1964, 'colors': ['red', 'white',
'blue']}
```

In [44]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
print(type(thisdict))
```

```
<class 'dict'>
```

In [45]:
```
thisdict=dict(name="john",age=36,country="norway")
print(thisdict)
```

```
{'name': 'john', 'age': 36, 'country': 'norway'}
```

In [46]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
x=thisdict["model"]
print(x)
```

```
mustang
```

In [47]:
```
x=thisdict.get("model")
print(x)
```

```
mustang
```

In [48]:
```
x=thisdict.keys()
print(x)
```

```
dict_keys(['brand', 'model', 'year'])
```

In [49]:
```
car={"brand":"ford","model":"mustang","year":1964}
x=car.keys()
print(x)
car["color"]="white"
print(x)
```

```
dict_keys(['brand', 'model', 'year'])
dict_keys(['brand', 'model', 'year', 'color'])
```

In [50]:
```
x=thisdict.values()
print(x)
```

```
dict_values(['ford', 'mustang', 1964])
```

In [51]:
```
car={"brand":"ford","model":"mustang","year":1964}
x=car.values()
print(x)
car["year"]=2020
print(x)
```

```
dict_values(['ford', 'mustang', 1964])
dict_values(['ford', 'mustang', 2020])
```

In [55]:
```
car={"brand":"ford","model":"mustang","year":1964}
x=car.values()
print(x)
car["color"]="red"
print(x)
```

```
dict_values(['ford', 'mustang', 1964])
dict_values(['ford', 'mustang', 1964, 'red'])
```

In [53]:
```
x=thisdict.items()
print(x)
```

```
dict_items([('brand', 'ford'), ('model', 'mustang'), ('year', 1964)])
```

In [56]:
```
car={"brand":"ford","model":"mustang","year":1964}
x=car.items()
print(x)
car["year"]=2020
print(x)
```

```
dict_items([('brand', 'ford'), ('model', 'mustang'), ('year', 1964)])
dict_items([('brand', 'ford'), ('model', 'mustang'), ('year', 2020)])
```

In [57]:
```
car={"brand":"ford","model":"mustang","year":1964}
x=car.items()
print(x)
car["color"]="red"
print(x)
```

```
dict_items([('brand', 'ford'), ('model', 'mustang'), ('year', 1964)])
dict_items([('brand', 'ford'), ('model', 'mustang'), ('year', 1964), ('color',
'red')])
```

In [59]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
if "model" in thisdict:
    print("yes,'model' is one of the keys in the thisdict dictionary")
```

```
yes,'model' is one of the keys in the thisdict dictionary
```

In [61]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
thisdict["year"]=2018
print(thisdict)
```

```
{'brand': 'ford', 'model': 'mustang', 'year': 2018}
```

In [62]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
thisdict.update({"year":2020})
print(thisdict)
```

```
{'brand': 'ford', 'model': 'mustang', 'year': 2020}
```

In [63]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
thisdict["color"]="red"
print(thisdict)
```

```
{'brand': 'ford', 'model': 'mustang', 'year': 1964, 'color': 'red'}
```

In [64]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
thisdict.update({"color":"red"})
print(thisdict)
```

```
{'brand': 'ford', 'model': 'mustang', 'year': 1964, 'color': 'red'}
```

In [65]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
thisdict.pop("model")
print(thisdict)
```

```
{'brand': 'ford', 'year': 1964}
```

In [66]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
thisdict.popitem()
print(thisdict)
```

```
{'brand': 'ford', 'model': 'mustang'}
```

In [67]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
del thisdict["model"]
print(thisdict)
```

```
{'brand': 'ford', 'year': 1964}
```

In [69]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
del thisdict
```

In [70]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
thisdict.clear()
print(thisdict)
```

```
{}
```

In [71]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
for x in thisdict:
    print(x)
```

```
brand
model
year
```

In [72]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
for x in thisdict:
    print(thisdict[x])
```

```
ford
mustang
1964
```

In [73]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
for x in thisdict.values():
    print(x)
```

```
ford
mustang
1964
```

In [74]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
for x in thisdict.keys():
    print(x)
```

```
brand
model
year
```

In [76]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
for x in thisdict.items():
    print(x)
```

```
('brand', 'ford')
('model', 'mustang')
('year', 1964)
```

In [77]:
```
thisdict={"brand":"ford","model":"mustang","year":1964}
mydict=dict(thisdict)
print(mydict)
```

```
{'brand': 'ford', 'model': 'mustang', 'year': 1964}
```

In [78]:
```
myfamily={"child1":{"name":"emil","year":2004},"child2":{"name":"linus","year"
print(myfamily)
```

```
{'child1': {'name': 'emil', 'year': 2004}, 'child2': {'name': 'linus', 'year':
2011}}
```

In [79]:
```
child1={"name":"emil","year":2004}
child2={"name":"tobias","year":2007}
child3={"name":"linus","year":2011}
myfamily={"child1":child1,"child2":child2,"child3":child3}
print(myfamily)
```

```
{'child1': {'name': 'emil', 'year': 2004}, 'child2': {'name': 'tobias', 'year':
2007}, 'child3': {'name': 'linus', 'year': 2011}}
```

In [80]:
```
print(myfamily["child2"]["name"])
```

```
tobias
```

In [81]:
```
for x,obj in myfamily.items():
    print(x)
for y in obj:
    print(y+':',obj[y])
```

```
child1
child2
child3
name: linus
year: 2011
```

practise questions

Write a Python program to create a list of 10 integers entered by the user and then display the list in reverse order without using the reverse() method or slicing [:: -1].

In [2]:
```
# Create an empty list
numbers = []

# Input 10 integers from the user
print("Enter 10 integers:")
for i in range(10):
    num = int(input(f"Enter number {i+1}: "))
    numbers.append(num)

# Display the list in reverse order without reverse() or slicing
print("\nList in reverse order:")
for i in range(len(numbers)-1, -1, -1):
    print(numbers[i], end=" ")
```

```
Enter 10 integers:
List in reverse order:
0 1 2 3 4 5 6 7 8 9
```

Write a Python program that takes a list of numbers as input and prints a new list containing only the even numbers from the original list.

In [ ]:
```
```

In [5]:
```
# Take list input from the user
numbers = list(map(int, input("Enter numbers separated by spaces: ").split()))

# Create a new list with only even numbers
even_numbers = []

for num in numbers:
    if num % 2 == 0:
        even_numbers.append(num)
```

```
# Display the even numbers list
print("Even numbers:", even_numbers)
```

```
Even numbers: [8, 6, 4, 2, 0]
```

Write a Python program to find and print the largest and smallest element in a list without using built-in max() and min() functions.

In [7]:
```
# Take list input from the user
numbers = list(map(int, input("Enter numbers separated by spaces: ").split()))

# Assume the first element is both smallest and largest
smallest = numbers[0]
largest = numbers[0]

# Traverse the list to find smallest and largest manually
for num in numbers:
    if num < smallest:
        smallest = num
    if num > largest:
        largest = num

# Print results
print("Smallest element:", smallest)
print("Largest element:", largest)
```

```
Smallest element: 3
Largest element: 7
```

Write a Python program that accepts a list of strings and prints the list a er removing all duplicate entries (preserve the original order of first occurrence).

In [8]:
```
# Take list of strings from the user
strings = input("Enter strings separated by spaces: ").split()

# List to store unique strings in original order
unique_strings = []

# Add only the first occurrence of each string
for s in strings:
    if s not in unique_strings:
        unique_strings.append(s)
```

```
# Print the result
print("List after removing duplicates:", unique_strings)
```

```
List after removing duplicates: ['how', 'are', 'you']
```

Write a Python program that takes 8 numbers as input from the user, stores them in a tuple, and then prints the sum, maximum, minimum, and average of the numbers.

In [9]:
```
# Input 8 numbers from the user
nums = []
print("Enter 8 numbers:")

for i in range(8):
    n = float(input(f"Enter number {i+1}: "))
    nums.append(n)

# Convert list to tuple
numbers = tuple(nums)

# Calculate results
total = 0
maximum = numbers[0]
minimum = numbers[0]

for num in numbers:
    total += num
    if num > maximum:
        maximum = num
    if num < minimum:
        minimum = num

average = total / len(numbers)

# Display the results
print("Tuple:", numbers)
print("Sum:", total)
print("Maximum:", maximum)
print("Minimum:", minimum)
print("Average:", average)
```

```
Enter 8 numbers:
Tuple: (7.0, 9.0, 6.0, 3.0, 5.0, 2.0, 4.0, 1.0)
Sum: 37.0
Maximum: 9.0
Minimum: 1.0
Average: 4.625
```

Write a Python program to create a tuple containing names of 5 students. Then accept a name from the user and check if it exists in the tuple. Print appropriate messages.

In [1]:
```python
# Create a tuple with 5 student names
students = ("Alice", "Bob", "Charlie", "David", "Eva")

# Accept a name from the user
name = input("Enter the name of the student: ")

# Check if the name exists in the tuple
if name in students:
    print(f"Yes, {name} is in the student list.")
else:
    print(f"Sorry, {name} is not in the student list.")
```

```
Yes, Eva is in the student list.
```

Write a Python program that creates two tuples and prints a new tuple containing elements common to both tuples (without duplicates).

In [2]:
```python
# Create two tuples
tuple1 = (1, 2, 3, 4, 5, 6)
tuple2 = (4, 5, 6, 7, 8, 9)

# Find common elements using set intersection
common_elements = tuple(set(tuple1) & set(tuple2))

# Print the result
print("Tuple 1:", tuple1)
print("Tuple 2:", tuple2)
print("Common elements (without duplicates):", common_elements)
```

```
Tuple 1: (1, 2, 3, 4, 5, 6)
Tuple 2: (4, 5, 6, 7, 8, 9)
Common elements (without duplicates): (4, 5, 6)
```

Write a Python program to accept 6 integers from the user and store them in a set. Then remove all even numbers from the set and print the remaining set.

In [3]:
```python
# Accept 6 integers from the user and store them in a set
numbers = set()

print("Enter 6 integers:")
for i in range(6):
    num = int(input(f"Enter integer {i+1}: "))
    numbers.add(num)

# Remove all even numbers from the set
numbers = {n for n in numbers if n % 2 != 0}

# Print the remaining set
print("Set after removing even numbers:", numbers)
```

```
Enter 6 integers:
Set after removing even numbers: {9, 3}
```

Write a Python program that takes two sets of integers as input from the user and prints Union, Intersec on, Difference (first – second) and Symmetric difference

In [4]:
```python
# Accept two sets of integers from the user
print("Enter integers for the first set (separated by spaces):")
set1 = set(map(int, input().split()))

print("Enter integers for the second set (separated by spaces):")
set2 = set(map(int, input().split()))

# Perform set operations
union_set = set1 | set2
intersection_set = set1 & set2
difference_set = set1 - set2
symmetric_difference_set = set1 ^ set2

# Print results
print("\nResults:")
```

```
print("Union:", union_set)
print("Intersection:", intersection_set)
print("Difference (First - Second):", difference_set)
print("Symmetric Difference:", symmetric_difference_set)
```

```
Enter integers for the first set (separated by spaces):
Enter integers for the second set (separated by spaces):
Results:
Union: {1, 2, 3, 5, 6, 7}
Intersection: set()
Difference (First - Second): {2, 3, 5}
Symmetric Difference: {1, 2, 3, 5, 6, 7}
```

Write a Python program to create a set of 10 random numbers between 1 and 20, then print whether the set contains the number 15 or not.

In [5]:
```
import random

# Create a set of 10 random numbers between 1 and 20
numbers = set(random.randint(1, 20) for _ in range(10))

# Print the generated set
print("Generated set:", numbers)

# Check if 15 is in the set
if 15 in numbers:
    print("Yes, the set contains the number 15.")
else:
    print("No, the set does not contain the number 15.")
```

```
Generated set: {4, 7, 13, 15, 16, 18, 19}
Yes, the set contains the number 15.
```

Write a Python program that accepts a sentence from the user and creates a set of all unique vowels (a, e, i, o, u) present in the sentence (case-insensi ve).

In [6]:
```
# Accept a sentence from the user
sentence = input("Enter a sentence: ")

# Define the set of vowels
```

```
vowels = set("aeiou")

# Convert sentence to lowercase and find unique vowels present
found_vowels = {ch for ch in sentence.lower() if ch in vowels}

# Print the result
print("Unique vowels present in the sentence:", found_vowels)
```

```
Unique vowels present in the sentence: {'o', 'e', 'u', 'a'}
```

Write a Python program to create a dic onary of 5 students where key is the student’s name and value is their mark. Then print the names of students who scored more than 75 marks.

In [7]:
```
# Create a dictionary of 5 students with their marks
students = {
    "Alice": 82,
    "Bob": 67,
    "Charlie": 90,
    "David": 74,
    "Eva": 88
}

# Print students who scored more than 75
print("Students who scored more than 75 marks:")
for name, mark in students.items():
    if mark > 75:
        print(name)
```

```
Students who scored more than 75 marks:
Alice
Charlie
Eva
```

Write a Python program that takes a dic onary of items and their prices, then calculates and prints the total bill amount. Example input: `{'Apple': 50, 'Banana': 20, 'Milk': 40}` → Total = 110

In [8]:
```python
# Accept a dictionary of items and their prices
items = {'Apple': 50, 'Banana': 20, 'Milk': 40}

# Calculate the total bill amount
total = sum(items.values())

# Print the result
print("Items:", items)
print("Total bill amount =", total)
```

```
Items: {'Apple': 50, 'Banana': 20, 'Milk': 40}
Total bill amount = 110
```

. Write a Python program to count the frequency of each character in a given string and store the result in a dic onary. Example: "hello" → `{'h':1, 'e':1, 'l':2, 'o':1}`

In [9]:
```python
# Accept a string from the user
text = input("Enter a string: ")

# Create an empty dictionary to store character frequencies
char_freq = {}

# Loop through each character in the string
for ch in text:
    if ch in char_freq:
        char_freq[ch] += 1   # Increment count if character already exists
    else:
        char_freq[ch] = 1    # Initialize count if character is new

# Print the result
print("Character frequency dictionary:", char_freq)
```

```
Character frequency dictionary: {'a': 1, 'p': 2, 'l': 1, 'e': 1}
```

Write a Python program to create a dic onary where keys are numbers from 1 to 10 and values are squares of the keys. Then print the dic onary in sorted order of keys.

In [10]:
```python
# Create a dictionary with keys 1 to 10 and values as their squares
squares = {x: x**2 for x in range(1, 11)}

# Print the dictionary in sorted order of keys
print("Dictionary of squares (sorted by keys):")
for key in sorted(squares.keys()):
    print(f"{key}: {squares[key]}")
```

```
Dictionary of squares (sorted by keys):
1: 1
2: 4
3: 9
4: 16
5: 25
6: 36
7: 49
8: 64
9: 81
10: 100
```

Write a Python program that accepts roll number and name of students and stores them in a dic onary. Keep accep ng un l the user enters "stop" as roll number. Finally print the en re phonebook-like dic onary.

In [14]:
```python
# Create an empty dictionary to store student data
students = {}

print("Enter student roll numbers and names.")
print("Type 'stop' as roll number to finish.\n")

while True:
    roll = input("Enter roll number: ")
```

```
    # Check for stop condition
    if roll.lower() == "stop":
        break

    name = input("Enter student name: ")
    students[roll] = name   # Store roll number as key and name as value

# Print the entire dictionary like a phonebook
print("\nStudent Dictionary (Phonebook style):")
for roll, name in students.items():
    print(f"Roll No: {roll} -> Name: {name}")
```

```
Enter student roll numbers and names.
Type 'stop' as roll number to finish.

Student Dictionary (Phonebook style):
Roll No: 15 -> Name: spidey
```

Write a Python program to convert a list of tuples `[('A', 65), ('B', 66), ('C', 67)]` into a dictionary and then print the dictionary.

In [13]:
```
# List of tuples
tuple_list = [('A', 65), ('B', 66), ('C', 67)]

# Convert list of tuples into a dictionary
dictionary = dict(tuple_list)

# Print the dictionary
print("Converted Dictionary:", dictionary)
```

```
Converted Dictionary: {'A': 65, 'B': 66, 'C': 67}
```

In [ ]:
```

```